Composition Pattern

Justification

The design pattern chosen for the Book Catalog module is the Composition pattern (also known as the Composite pattern).

The Book Catalog in the Library Management System is not a flat list. The catalog must organize books into logical groupings: by classification, by subject, by acquisition batch, or by physical section of the library. A single BookCatalog can contain individual Book entries, and at the same time it can contain sub-catalogs that are themselves full BookCatalog objects with their own books inside them. The system must be able to treat a single book and an entire catalog section with the same interface, for example when performing a search or generating a display. The Composition pattern is designed for exactly this kind of part-whole hierarchy.

The CatalogComponent interface defines the common contract: getTitle(), display(), search(), and getDetails(). The Book class is the leaf — it implements CatalogComponent directly and represents a single indivisible entry. The BookCatalog class is the composite — it also implements CatalogComponent, but internally it holds a list of CatalogComponent objects. Because both Book and BookCatalog implement the same interface, a recursive search across the entire hierarchy is a single method call on the root catalog. The caller does not need to know whether it is traversing a flat list or a nested structure.

This structure also reflects the real requirement from BR_LMS_01, which states that the system must maintain a digital catalog containing a record for every book and material. BR_LMS_03 requires that books be organized according to a standardized classification system, which maps naturally to nested sub-catalogs. BR_LMS_04 allows staff to update catalog metadata, which the composite handles through the updateMetadata() method propagating through the tree.

The professor suggested Composition as the appropriate pattern here after the team proposed Singleton. Singleton would guarantee a single catalog instance, but it does not address the structural problem of organizing books hierarchically. Composition solves the structural problem and the uniqueness constraint can be handled separately at the application layer if needed, keeping each class responsible for one concern. This is consistent with the Single Responsibility Principle.

The Open/Closed Principle is also satisfied: if the library later needs to introduce a DigitalSection or an ArchiveSection, a new class implementing CatalogComponent is added

without modifying Book, BookCatalog, or any code that traverses the catalog. The existing hierarchy simply gains a new node type.

UML Class Diagram